

Metodi Random Forest e XGBoost (e Gradient Boosting Machines)

In questa lezione vedremo con maggiore dettaglio due metodi di classificazione/regressione

1. Random Forest

Random Forest è un algoritmo di **ensemble learning** basato su moltissimi **alberi decisionali**.

Idea principale

- Si creano tanti alberi decisionali **indipendenti**
- Ognuno vede solo una **sottocampionatura casuale** dei dati (bootstrap sampling = bagging)
- Ognuno vede solo un **sottoinsieme casuale** delle features ad ogni split (random feature selection)

Previsione finale

- **Classificazione** → voto di maggioranza (modalità)
- **Regressione** → media dei valori predetti

Vantaggi principali

- Molto robusto al rumore e agli outlier
- Resiste bene all'overfitting (rispetto a un singolo albero)
- Non richiede molta preprocessazione (gestisce bene variabili numeriche e categoriche)
- Fornisce una stima naturale dell'**importanza delle variabili**

Svantaggi

- Molto più lento e pesante in memoria rispetto a un singolo albero
- Le predizioni non sono interpretabili come un singolo albero
- Può essere meno performante di gradient boosting su dataset strutturati

2. XGBoost (Extreme Gradient Boosting)

XGBoost è una implementazione molto ottimizzata e potenziata del metodo **Gradient Boosting Machines** (GBM).

Idea di base del Gradient Boosting

Si costruiscono alberi **sequenzialmente**:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

dove:

- $F_m(x)$ = modello finale dopo m iterazioni
- $h_m(x)$ = albero debole che corregge gli errori residui del modello precedente
- η = **learning rate** (shrinkage)

Cosa rende XGBoost speciale

Caratteristica	Descrizione
Regularizzazione L1 + L2	Penalizza la complessità degli alberi → meno overfitting
Gestione automatica dei missing	Decide autonomamente dove mandare i valori mancanti
Approssimazione di secondo ordine	Usa gradiente e hessiano (curvatura) → convergenza più veloce
Split finding ottimizzato	Algoritmo approssimato + histogram-based → molto più veloce
Cache-aware & parallelizzazione	Ottimizzazioni a livello di CPU/cache
Early stopping	Interrompe se non migliora sulla validazione
Feature importance & SHAP support	Ottime interpretazioni post-hoc

Confronto sintetico

Aspetto	Random Forest	XGBoost
Tipo di ensemble	Bagging (parallelo)	Boosting (sequenziale)
Alberi indipendenti?	Sì	No – ognuno corregge i precedenti
Velocità di addestramento	Più veloce (parallelo)	Più lento (sequenziale)
Prestazioni tipiche	Molto buone	Spesso migliori su dati tabulari
Robustezza al rumore	Ottima	Buona (con regolarizzazione)
Sensibilità a iperparametri	Media	Alta
Overfitting	Difficile	Possibile (serve tuning)

Regola empirica

- Dataset **piccolo / medio** e vuoi velocità → **Random Forest**
- Dataset **strutturato** (tabellare) e vuoi il **massimo risultato possibile** → **XGBoost** / LightGBM / CatBoost
- Hai poco tempo per fare tuning → **Random Forest** o **LightGBM** con parametri di default ragionevoli

Qualche spiegazione aggiuntiva su Gradient Boosting Machines (GBM)

Gradient Boosting è una delle tecniche di **machine learning ensemble** più potenti e utilizzate al mondo, specialmente su dati tabellari (strutturati).

È la base di librerie molto famose come **XGBoost**, **LightGBM**, **CatBoost**, **HistGradientBoosting** di scikit-learn.

Idea intuitiva di base

Partiamo da un modello molto semplice (spesso un albero decisionale debole, depth 3–6) e lo miglioriamo **iterativamente** correggendo gli errori che commette.

A differenza di Random Forest (che costruisce alberi indipendenti in parallelo), nel Gradient Boosting:

- gli alberi sono **sequenziali**
- ogni nuovo albero cerca di correggere **gli errori residui** del modello precedente
- si fa **gradient descent** nello spazio delle funzioni (non nei parametri!)

Risultato: si ottiene un modello **molto espressivo** che spesso batte quasi tutti gli altri metodi su dataset tabellari (fino all'arrivo dei transformer su certi compiti).

Concetto chiave: pseudo-residuals

Invece di prevedere direttamente y , ogni nuovo albero prevede **quanto il modello attuale sbaglia** (il gradiente negativo della loss).

Per la regressione con **Mean Squared Error** (MSE):

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

Il **gradiente** (derivata rispetto a $\hat{y}$) è:

$$-\frac{\partial L}{\partial \hat{y}} = y - \hat{y} \quad \Rightarrow \quad \text{pseudo-residual} = r_i = y_i - F_{m-1}(x_i)$$

Quindi: **ogni nuovo albero viene fittato sui residui** (pseudo-residuals) del modello cumulativo precedente.

Algoritmo generale di Gradient Boosting

Pseudocodice classico (Friedman 2001):

Perché si chiama “Gradient Descent in function space”?

Nel gradient descent classico:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$$

In Gradient Boosting:

$$F'(x) \leftarrow F(x) - \underbrace{\eta \cdot (\text{nuovo albero che approssima il gradiente})}_{\text{direzione di discesa}}$$

Ma invece di modificare parametri θ , aggiungiamo **una nuova funzione** (l'albero) che punta nella direzione opposta al gradiente della loss **nello spazio delle funzioni**.

Questo è il motivo per cui si dice che GBM fa **functional gradient descent**.

Iperparametri principali da regolare

Parametro	Tipico range	Effetto principale	Consiglio pratico
n_estimators (M)	100–5000	Più alberi → più capacità (ma rischio overfit)	Usa early stopping!
learning_rate (η)	0.01 – 0.3	Più basso → apprendimento lento ma stabile	Inizia da 0.1, poi abbassa e a
max_depth	3–10	Profondità degli alberi	3–6 molto comune
subsample	0.6–1.0	Frazione di dati usata per ogni albero	0.8 spesso ottimo (stochasti
colsample_bytree	0.6–1.0	Frazione di feature per split	aiuta contro overfitting
min_child_weight	1–10	Regolarizzazione (XGBoost/LightGBM)	valori più alti → alberi più se
gamma / reg_lambda	0–5	Regolarizzazione L1/L2 sugli split	utile per evitare split inutili

In sintesi – punti di forza e di debolezza

Pro

- Spessissimo **stato dell'arte** su dati tabellari (2020–2025)
- Gestisce bene missing values (soprattutto XGBoost, LightGBM, CatBoost)
- Feature importance naturale
- Molto flessibile (qualsiasi loss differenziabile)

Contro

- Sensibile al tuning (molti iperparametri)
- Può overfittare facilmente senza regolarizzazione / early stopping
- Addestramento sequenziale → non parallelo come Random Forest
- Peggior di modelli deep su dati non strutturati (immagini, testo)

Inizia a programmare o genera codice con l'IA.

Il problema dell'interpretabilità dei modelli "black box"

Il problema dell'interpretabilità dei modelli è centrale in tutti i modelli "black box".

In molti ambiti la previsione è fondamentale ma è necessario (alcune volte anche obbligatorio) associare anche una interpretazione del modello.

Un utile riferimento: <https://christophm.github.io/interpretable-ml-book/>

Importanza delle variabili e SHAP Summary Plot

Quando usiamo modelli complessi (Random Forest, XGBoost, LightGBM, reti neurali, ecc.) spesso vogliamo capire **quali variabili contano di più** per le previsioni.

Esistono diversi metodi per stimare l'**importanza delle feature**. I più usati sono:

1. **Feature Importance nativa** del modello
(es. gain, cover, weight in XGBoost / split importance in Random Forest)
2. **Permutation Importance**
(misura quanto peggiora il modello se mescoliamo casualmente una variabile)
3. **SHAP values** (SHapley Additive exPlanations)
Metodo più moderno, teorico e coerente → oggi considerato lo **standard de facto** per interpretabilità

Perché SHAP è speciale?

SHAP si basa sulla teoria dei giochi (valori di Shapley) e risponde alla domanda:

“Quanto contribuisce ciascuna variabile alla previsione di questo singolo esempio?”

e poi aggrega queste informazioni per capire il comportamento **globale** del modello.

SHAP Summary Plot

Il **SHAP summary plot** (beeswarm plot) mostra:

- **asse x** → impatto medio sulla previsione (SHAP value)
valori positivi → aumentano la previsione
valori negativi → la diminuiscono
- **asse y** → le variabili ordinate per importanza media assoluta
- **ogni punto** = un'osservazione del dataset
colore = valore della feature (di solito rosso = alto, blu = basso)

Esempio tipico di lettura:

- Età alta (rosso) → $\text{SHAP} > 0$ → aumenta molto la probabilità di malattia
- Reddito basso (blu) → $\text{SHAP} < 0$ → diminuisce la probabilità
- Variabile quasi verticale → poca variabilità di impatto → importanza bassa

Codice rapido per generarlo (XGBoost o LightGBM)

```
import shap
```

```
# Supponiamo di avere già il modello addestrato: model
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X)

# Summary plot (beeswarm)
shap.summary_plot(shap_values, X, plot_type="dot", max_display=15)

# Versione a barre (importanza media assoluta)
shap.summary_plot(shap_values, X, plot_type="bar")
```

✓ Esercitazione in vista dell'esame.

Esempi di domande:

1. Spiega in 10 righe l'algoritmo della discesa del gradiente (5 min)
2. Quali misure di performance sono più adatte nel caso di dataset sbilanciati e perché
3. In che cosa consiste il problema dello XOR e come è stato risolto
4. Spiega l'algoritmo di retropropagazione e come si collega alle reti neurali
5. Spiega l'algoritmo Random Forest e come questo algoritmo modifica un semplice albero di decisioni

Inizia a programmare o genera codice con l'IA.